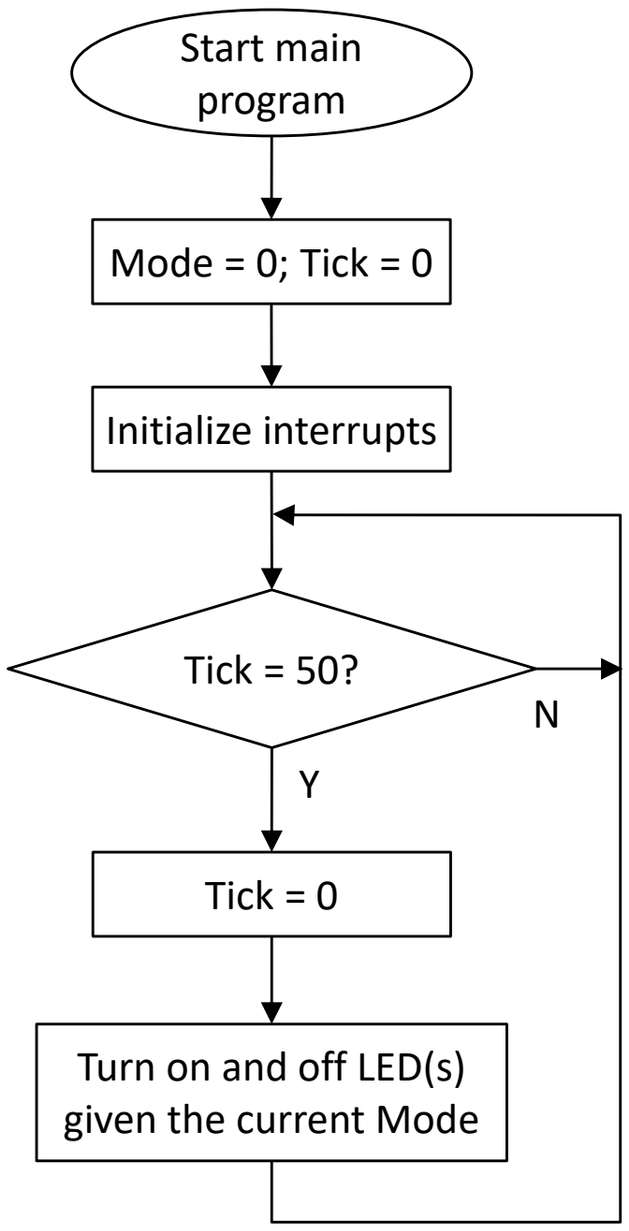
EECE3264 Lab06 StateChart



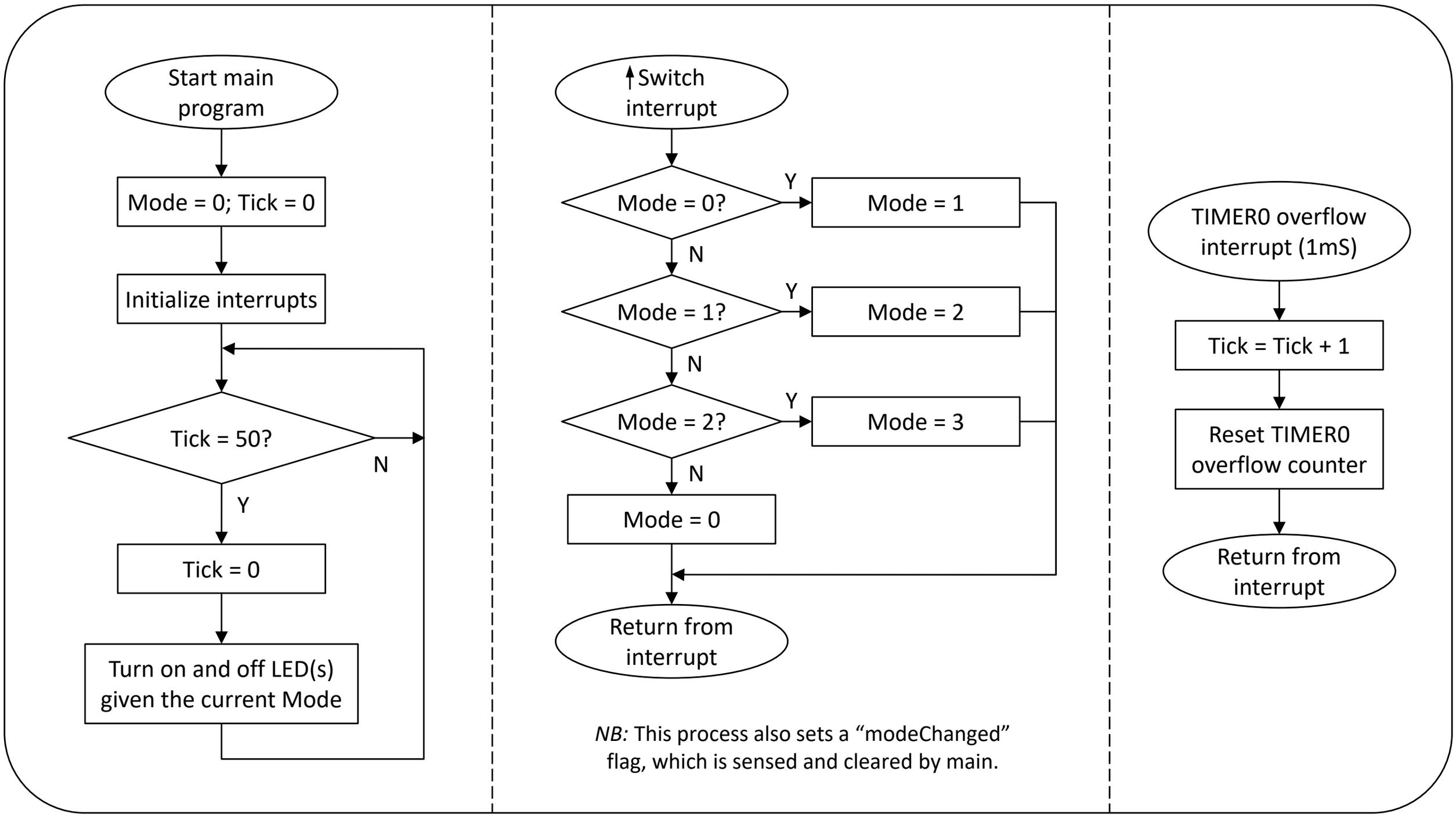
NB: This process also sets a "modeChanged" flag, which is sensed and cleared by main.

The main loop ***reads*** **Mode** and **Tick** (and sometimes ***resets*** **Tick** to zero)

The Switch interrupt code ***changes*** **Mode**

The TIMER0 code ***increments*** **Tick** once every millisecond

EECE3264 Lab06 StateChart



The main loop ***reads*** **Mode** and **Tick** (and sometimes ***resets*** **Tick** to zero)

The Switch interrupt code ***changes*** **Mode**

The TIMER0 code ***increments*** **Tick** once every millisecond

Handling Interrupts in C

- AVR Studio provides good support for interrupt processing in C
  - ISR handler must be named "ISR"
  - Argument must be a specific interrupt vector name as defined in the `iom128a.h` file
  - `iom128a.h` also defines register names and bit operators, too—very helpful!

```
ISR(INT1_vect)
{
   ...
}
```

Tip: While programming, R-click on a defined constant, macro, variable, or function name and choose "Goto Implementation"—Atmel Studio will find and open the source definition for you!

A State-Driven Interrupt Handler Framework (1 of 3)

```
#include <avr/io.h>                 // standard AVR header
#include <avr/interrupt.h>          // needed for interrupts
#include <avr/delay.h>              // needed to use _delay_ms()

#define DELAY 50

// enumerate the possible patterns
enum patterns
{
        RightToLeft,                // E0 to E7
        LeftToRight,                // E7 to E0
        BackAndForth,               // E0 to E7 to E0
        MysteryPattern              // ???
};

// global variable to keep track of the current LED pattern
enum patterns Mode;

int main(void)
{
    ...
```

NB: You're ***not*** **allowed** to use **`_delay_ms()`** in Lab06!! (Use a **tick-timer** instead)

A State-Driven Interrupt Handler Framework (2 of 3)

```
int main(void)
{
    // declare variables, initialize HW registers (interrupts,
    // ports, etc.) and initialize Mode to starting pattern

    // enable interrupts globally
    sei();

    while(1) // loop forever
    {
        // execute code based on the current pattern
        switch(Mode)
        {
            case(RightToLeft):
                // turn off current LED, turn on next LED in the pattern
                break;
            case(LeftToRight):
                :
        }
        _delay_ms(DELAY);  // leave the LED on for 50mS
    }
}
```

Switch-Case statements are more compact than if-then-else statements

Again, don't use **`_delay_ms()`** in Lab06. Instead, wait here until "Tick" reaches 50 (incremented automatically via the timer ISR). Once it reaches 50, reset it to zero and continue looping.

A State-Driven Interrupt Handler Framework (3 of 3)

```
ISR(INTxx_vect)
{
   // Invoked when joystick button PB4 (which must be connected
   // externally to a pin that can cause interrupts) is released

   // change LED blinking pattern
   switch(Mode)
   {
      case(RightToLeft):
         Mode = LeftToRight;
         break;
      case(LeftToRight):
         Mode = BackAndForth;
         break;
        ⋮
   }
}
```

NB: This is **NOT the correct interrupt number for Lab06**!